Worksheet — 1.4 Data Types, Data Structures & Boolean Algebra — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Activity 1 — Conversions

#	Denary	Binary (8-bit)	Hex
1	**217**	**11011001**	**D9**
2	**173**	`10101101`	**AD**
3	**90**	**01011010**	`5A`
4	**202**	**11001010**	**CA**
5	**47**	`00101111`	**2F**
6	**190**	**10111110**	**BE**
7	**45**	**00101101**	`2D`
8	**179**	`10110011`	**B3**

Worked examples: - **217**: 128+64=192, +16=208, +8=216, +1=217 → `11011001`. Nibbles `1101 1001` = D9. - **`10101101`**: 128+32+8+4+1 = 173. Nibbles `1010 1101` = AD. - **`5A`**: 5=`0101`, A=`1010` → `01011010` = 64+16+8+2 = 90. - **202**: 128+64=192, +8=200, +2=202 → `11001010` = `1100 1010` = CA. - **`00101111`**: 32+8+4+2+1 = 47. `0010 1111` = 2F. - **190**: 128+32=160, +16=176, +8=184, +4=188, +2=190 → `10111110` = BE. - **`2D`**: 2=`0010`, D=`1101` → `00101101` = 32+8+4+1 = 45. - **`10110011`**: 128+32+16+2+1 = 179. `1011 0011` = B3.

Activity 2 — Two's complement

(a)

Denary	8-bit two's complement	Working
−20	**11101100**	+20 = `00010100`; invert `11101011`; +1 = `11101100`
−100	**10011100**	+100 = `01100100`; invert `10011011`; +1 = `10011100`
−1	**11111111**	+1 = `00000001`; invert `11111110`; +1 = `11111111`

Check −20: −128+64+32+8+4 = −20 ✓. −100: −128+16+8+4 = −100 ✓.

(b)(i) 35 − 18 = 17

```
 A   = 00100011   (35)
-B   = 11101110   (−18: +18=00010010 → invert 11101101 → +1 = 11101110)
----------------
       1 00010001   carry out of MSB = 1 → DISCARD
Sum  = 00010001
```

Denary result = **+17** (MSB 0 = positive; 16+1 = 17) ✓

(b)(ii) 20 − 50 = −30

```
 A   = 00010100   (20)
-B   = 11001110   (−50: +50=00110010 → invert 11001101 → +1 = 11001110)
----------------
Sum  = 11100010   (no carry out of MSB)
```

Denary result = **−30**. The MSB is **1**, so the result is negative. Check: −128+64+32+2 = −30 ✓.

Activity 3 — Floating-point normalisation

Given mantissa `0.0001101` (point after first bit). - **(a)** Move the point **3 places to the right** so the mantissa becomes `0.1101…` (first bit after the point is 1). - **(b)** Moving the point **right ⇒ exponent decreases**. - **(c)** Normalised mantissa = **`0.1101000`** (`0.1101` with trailing zero padding). - **(d)** Original exponent was effectively 0; moving right 3 places means we must **subtract 3** → exponent = **−3**. (Value = `0.0001101` × 2^0 = `0.1101` × 2^{-3}, unchanged.) - **(e)** Normalisation removes wasted leading zeros so the fixed-width mantissa holds the **maximum number of significant figures → greatest precision** (and gives a unique representation).

Activity 4 — Binary shifts

(a) `00010110` logical left 2 = **`01011000`**. Effect: shift **left**, multiplies by **2^2 = ×4**. (Check: 22 × 4 = 88 = `01011000` ✓.)

(b) `10110000` logical right 3 = **`00010110`**. Effect: shift **right**, divides by **2^3 = ÷8** (integer division; bits shifted off are lost, 0s filled from the left). (Check: 176 ÷ 8 = 22 = `00010110` ✓.)

Activity 5 — Data structures

(a) 1→**B**, 2→**G**, 3→**J**, 4→**C**, 5→**F**, 6→**H**, 7→**A**, 8→**D**, 9→**I**, 10→**E**.

(b) Trace table (stack = LIFO, removes the **top/most recent**; queue = FIFO, removes the **front/oldest**):

Step	Operation	Stack (top → bottom)	Stack removed	Queue (front → rear)	Queue removed
1	add 7	7	–	7	–
2	add 3	3, 7	–	7, 3	–
3	add 9	9, 3, 7	–	7, 3, 9	–
4	remove	3, 7	**9**	3, 9	**7**
5	add 5	5, 3, 7	–	3, 9, 5	–
6	remove	3, 7	**5**	9, 5	**3**

Activity 6 — Boolean

(a) XOR

A	B	Q
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

NAND

A	B	Q
0	0	**1**
0	1	**1**
1	0	**1**
1	1	**0**

(b)(i) `¬(A · B) + B` = `(¬A + ¬B) + B` (De Morgan's on the bracket) = `¬A + (¬B + B)` (associative/commutative) = `¬A + 1` (complement: ¬B + B = 1) = **`1`** (null law). The expression is always true.

(b)(ii) `¬(¬A + ¬B)` = `¬(¬A) · ¬(¬B)` (De Morgan's: break the bar, swap OR→AND) = `A · B` (double negation on each term) = **`A · B`**.

(c) Half adder

A	B	Sum	Carry
0	0	**0**	**0**
0	1	**1**	**0**
1	0	**1**	**0**
1	1	**0**	**1**

Sum = A ⊕ B Carry = A · B.

Activity 7 — Floating-point addition & normalisation

(a)

- **Step 1 — denary check.** P: mantissa `0.1010000` = 0.5 + 0.125 = 0.625; exponent `0011` = +3; value = 0.625 × 2^3 = **5**. (Equivalently, move the point 3 right: `0101.0000` = 5.) Q: mantissa `0.1000000` = 0.5; exponent `0001` = +1; value = 0.5 × 2^1 = **1**. (`01.000000` = 1.)
- **Step 2 — align exponents.** P's exponent is 3, Q's is 1, difference = 2, so shift Q's mantissa **right 2 places** and add 2 to its exponent: Q becomes mantissa **`0.0010000`**, exponent **`0011`**. (Check: `0.001` × 2^3 = `001.0` = 1 — value unchanged ✓.)
- **Step 3 — add the mantissas** (exponents now equal, so mantissas add directly):

``` 0.1010000 + 0.0010000

---

```
0.1100000
```

`` - **Step 4 — normalise.**0.1100000already starts0.1..., so it is **already normalised**. Result: mantissa **0.1100000**, exponent **0011`**. -** Step 5 — denary check. **$0.11_2$ = 0.75; 0.75 × $2^3$ =** 6** = 5 + 1 ✓.

**(b)** `0.0001100` × 2^`0101`: - Exponent `0101` = +5. Un-normalised value: move the point 5 right → `00011.00` = **3**. - Normalise: the point must move **3 places right** so the mantissa starts `0.1…` → mantissa **`0.1100000`**; the exponent decreases by 3: 5 − 3 = 2 → exponent **`0010`**. - Check: $0.11_2$ = 0.75; 0.75 × $2^2$ = **3** ✓ (value unchanged).

**(c)** `1.1101000` × 2^`0100`: - Exponent `0100` = +4. Mantissa `1.1101000` is negative (sign bit 1); its value = −1 + $0.1101_2$ = −1 + (0.5 + 0.25 + 0.0625) = −1 + 0.8125 = −0.1875. Un-normalised value = −0.1875 × $2^4$ = **−3**. - A normalised **negative** mantissa must start `1.0…`. Shift the mantissa **left 2 places**: `1.1101000` → `1.1010000` (still starts 1.1) → **`1.0100000`** ✓ starts `1.0`. The exponent decreases by 2: 4 − 2 = 2 → exponent **`0010`**. - Check: `1.0100000` = −1 + 0.25 = −0.75; −0.75 × $2^2$ = **−3** ✓ (value unchanged).

## Activity 8 — Karnaugh maps

**(a)** Q = ¬A·B·C + A·B·C + A·B·¬C. The map (1s at A=0/BC=11, A=1/BC=11, A=1/BC=10):

| | **BC=00** | **BC=01** | **BC=11** | **BC=10** |
|---|---|---|---|---|
| **A=0** | 0 | 0 | **1** | 0 |
| **A=1** | 0 | 0 | **1** | **1** |

Groups (pairs, overlapping at cell A=1/BC=11): - Vertical pair in column BC=11 (both rows): A changes, B=1 and C=1 stay → **B·C**. - Horizontal pair in row A=1, columns BC=11 and BC=10 (adjacent in Gray code): C changes, A=1 and B=1 stay → **A·B**.

**Simplified Q = B·C + A·B** (equivalently Q = B·(A + C)). *Check:* B·C covers ¬A·B·C and A·B·C; A·B covers A·B·C and A·B·¬C — exactly the three original minterms ✓.

**(b)** Groups: - **Quad of the four corners** (wrap-around both horizontally and vertically): cells (AB=00,CD=00), (AB=00,CD=10), (AB=10,CD=00), (AB=10,CD=10). In all four: B=0 and D=0 (A and C both change) → **¬B·¬D**. - **Pair** in row AB=01 at columns CD=01 and CD=11: A=0, B=1, D=1 stay (C changes) → **¬A·B·D**.

**Simplified Q = ¬B·¬D + ¬A·B·D.** *Check:* ¬B·¬D covers minterms 0, 2, 8, 10 (the corners); ¬A·B·D covers minterms 5 and 7 — exactly the six 1s in the map, groups are powers of 2 and as large as possible ✓.

## Challenge box

**(a)** `10000000` = the −128 column only set, all others 0 → **−128**. (It is not zero; zero is `00000000`.)

**(b)** In two's complement the MSB is a single negative weight, so each bit pattern maps to exactly one value; `00000000` is the only pattern that sums to 0. Sign-and-magnitude has both `00000000` (+0) and `10000000` (−0); two's complement avoids this, giving **one** zero (and one extra negative value, −128).
```

(c) Green 90_{16} = (9 × 16) + 0 = **144**. Blue FF_{16} = (15 × 16) + 15 = **255**. Hex is used because each pair of hex digits = exactly one byte (8 bits / one colour channel), making it a **compact, human-readable, lossless** shorthand for the underlying binary, far less error-prone than writing 24 raw bits.

Section B — model answers

Q1 [2]

Full-mark response: "ASCII uses 7 bits (commonly stored in one byte), giving only 128 characters, which is essentially limited to the English alphabet, digits, punctuation and control characters. Unicode uses more bits per character (e.g. up to 4 bytes in UTF-8), so it can represent over a million code points, covering virtually every world language's script plus symbols and emoji."

Mark points (a described difference needs both sides): - **[1]** ASCII: 7 bits / 128 characters / English-centric character set. - **[1]** Unicode: more bits per character (16/32-bit, or variable-length UTF-8), so far more characters — supports (almost) all languages/scripts/emoji. ASCII is a subset of Unicode (first 128 code points identical) is also creditworthy as part of the comparison.

Q2 [3]

Full-mark response (complete working):

1. +45 in 8-bit binary: 45 = 32 + 8 + 4 + 1 → `00101101`
2. Invert all bits: `11010010`
3. Add 1: `11010010` + `1` = **`11010011`**

So −45 is stored as **`11010011`**.

Verification: −128 + 64 + 16 + 2 + 1 = −128 + 83 = −45 ✓.

Mark points: - **[1]** Correct binary for +45: `00101101`. - **[1]** Correct method: invert all bits **and** add 1 (or equivalent, e.g. −128 place-value construction). - **[1]** Correct final answer `11010011`.

Q3 [4]

(i) [2] mantissa `01011000`, exponent `0011`

Full-mark response: - Mantissa = `0.1011000` (positive, sign bit 0). Exponent `0011` = +3. - Move the binary point 3 places to the right: `0.1011000` → `0101.1000`. - `0101.1` = 4 + 1 + 0.5 = **5.5**

Verification (fraction method): mantissa = 0.5 + 0.125 + 0.0625 = 0.6875; 0.6875 × 2^3 = 0.6875 × 8 = 5.5 ✓.

Mark points: **[1]** correct method (exponent = 3, point moved 3 right, or 0.6875 × 8); **[1]** answer **5.5**.

(ii) [2] mantissa `10110000`, exponent `0011`

Full-mark response: - Mantissa = `1.0110000` — the sign bit is 1, so the number is **negative**. Exponent `0011` = +3. - Move the binary point 3 places to the right (keeping two's complement): `1.0110000` → `1011.0000`. - `1011` in 4-bit two's complement = −8 + 2 + 1 = **−5**

Verification (fraction method): mantissa value = $-1 + 0.011_2$ = −1 + (0.25 + 0.125) = −0.625; −0.625 × 2^3 = −0.625 × 8 = −5 ✓.

Mark points: **[1]** recognises negative mantissa and applies a valid two's complement method with exponent 3; **[1]** answer **−5**.

Q4 [3]

Full-mark response:

```
X = A·B + A·¬B + ¬A·B
  = A·(B + ¬B) + ¬A·B     (distributive law – factor A from the first two terms)
  = A·1 + ¬A·B            (complement law: B + ¬B = 1)
  = A + ¬A·B              (identity law: A·1 = A)
  = (A + ¬A)·(A + B)      (distributive law of OR over AND)
  = 1·(A + B)             (complement law: A + ¬A = 1)
  = A + B                 (identity law)
```

X = A + B

Mark points: - **[1]** Factorises the first two terms: A·B + A·¬B = A·(B + ¬B) = A. - **[1]** Correctly reduces A + ¬A·B to A + B (via distribution/complement as shown, or by the absorption-based identity A + ¬A·B = A + B). - **[1]** Final answer **A + B**, with steps/rules shown.

Verification by truth table: (0,0)→0+0+0=0; (0,1)→¬A·B=1; (1,0)→A·¬B=1; (1,1)→A·B=1 — identical to A OR B ✓.

Q5 [3]

Full-mark response: "A stack is last-in-first-out, which exactly matches undo: the action to reverse must always be the **most recent** one performed. Each time the user performs an action, the editor **pushes** a description of that action onto the top of the stack. When the user presses undo, the editor **pops** the top item off the stack — which is guaranteed to be the most recent action — and reverses it; pressing undo again pops the next most recent, and so on."

Mark points: - **[1]** Stack is LIFO, matching the requirement that the most recent action is undone first. - **[1]** Performing an action → the action is **pushed** onto the (top of the) stack. - **[1]** Undo → the top action is **popped** off the stack and reversed (repeated undos work back through history in reverse order).

Q6 [2]

Full-mark response: "A D-type flip-flop stores the value of a single bit (a 1 or a 0), making it the basic building block of registers and memory. Its clock input synchronises the circuit: the output can only change to match the D input at a clock edge (e.g. the rising edge), so the stored bit is held constant between clock pulses."

Mark points: - **[1]** Purpose: stores/latches **one bit** (elemental memory/used in registers). - **[1]** Clock: output only updates on a clock pulse/edge — synchronises when the stored value can change; value is held between pulses.

Q7 [4]

Full-mark response: "To store a score, the hash function is applied to the player's name (the key); this calculates an index/address in the table, and the name and score are stored at that location. To retrieve a score, the same hash function is applied to the name again, giving the same index directly, so the score is

found in roughly constant time without searching through the other players. A collision occurs when two different names hash to the same index. It is handled by, for example, storing the second record in the next free slot (linear probing) — retrieval then checks slots in sequence from the hashed index until the matching key is found — or by keeping a linked list of all records at that index (chaining)."

Mark points: - **[1]** Store: hash function applied to the key (player name) computes the index/address where the record is placed. - **[1]** Retrieve: re-hashing the same name yields the same index, so lookup is direct/~O(1), no linear search needed. - **[1]** Collision: two different keys produce the **same index/address**. - **[1]** Valid resolution described: linear probing/next free location (with sequential check on retrieval) **or** chaining (list at that index) **or** rehashing to a second function/overflow area.

Q8* [9] — levels of response

Indicative content:

Hash table: - Lookup by name is ~**O(1)**: hash the name, go straight to the index — the fastest option for requirement 1. - But a hash table is inherently **unordered** (the hash function scatters keys), so producing the alphabetical list means extracting **all** entries and sorting them — **O(n log n)** — *every* time the app opens. - Collisions need handling (probing/chaining) and performance degrades as the table fills; the table may also need resizing/rehashing as contacts grow.

Binary search tree: - Lookup is **O(log n)** when the tree is balanced — slower than a hash table in theory, but for a realistic contacts list (even thousands of names) the difference is a handful of comparisons and imperceptible to the user. - An **in-order traversal** (left subtree, node, right subtree) visits nodes in key order, so the alphabetical list is produced directly in **O(n)** with **no sorting** — ideal for requirement 2. - Worst case: inserting names in already-sorted order degenerates the BST into a linked list, making lookup **O(n)** (mitigated by self-balancing trees); nodes carry pointer overhead.

Evaluation/recommendation: the hash table optimises only one requirement and is poor for the other; the BST satisfies **both** requirements well (near-instant lookup at contacts-app scale, and a free alphabetical ordering). Recommend the **BST** (ideally balanced). (A justified hybrid — hash table plus a separately maintained sorted structure, trading memory and duplication for speed — is also creditworthy.)

Levels: - **Level 3 (7–9 marks):** Detailed discussion of **both** structures against **both** requirements, using accurate technical content (hashing/collisions, O(1) vs O(log n), unordered nature of hash tables, in-order traversal, unbalanced-tree worst case). Clear, justified recommendation that follows logically from the comparison. Well-developed line of reasoning. - **Level 2 (4–6 marks):** Both structures considered with mostly accurate points, but the discussion may address one requirement superficially (e.g. omits *why* a hash table cannot give ordering cheaply, or never mentions in-order traversal); recommendation present but only partly justified. - **Level 1 (1–3 marks):** Basic, generic statements ("hash tables are fast", "trees store data in order") with errors or little development; one structure only, or no real link to the scenario; recommendation missing or unjustified. - **0 marks:** No creditworthy response.

Model Level-3 answer:

"A hash table would make the first requirement extremely fast: applying the hash function to the contact's name gives the index of their record directly, so lookup is approximately O(1) regardless of how many contacts are stored, with only occasional extra steps when collisions must be resolved by probing or chaining. However, a hash table stores records in effectively random positions, because a good hash function deliberately scatters keys. The app's second requirement — an alphabetical list every time it opens — would therefore require reading out all n entries and sorting them, an O(n log n) operation

repeated on every launch, and the table also needs collision handling and periodic resizing as the contact list grows.

A binary search tree stores each name so that everything in a node's left subtree precedes it alphabetically and everything in the right subtree follows it. Searching for a name halves the search space at each comparison, giving O(log n) lookup when the tree is reasonably balanced — for even 10,000 contacts that is about 14 comparisons, which is imperceptibly different from a hash table's O(1) for a user. Crucially, an in-order traversal (visit left subtree, node, right subtree) outputs every contact already in alphabetical order in O(n) time with no sorting at all, satisfying the second requirement directly. The BST's weakness is that inserting names in nearly sorted order can unbalance it towards an O(n) linked list, though a self-balancing variant removes this risk, and each node needs extra memory for its child pointers.

Weighing the two: the hash table is optimal for one requirement but poor for the other, while the BST is near-optimal for both — its lookup is fast enough at contacts-app scale, and it provides the alphabetical listing for free. I would recommend the developer use a binary search tree (ideally self-balancing), since it satisfies both stated requirements efficiently without the repeated sorting cost a hash table would impose every time the app opens."